

A Rough Introduction to Distributional Reinforcement Learning

Jonas Muller

April 27, 2026

Outline

- 1 Motivation
- 2 Core Idea
- 3 Main Algorithms
- 4 Practical View
- 5 Template Slides To Fill

Why Distributional RL?

- Classical RL tracks only the expected return: $Q(s, a) = \mathbb{E}[\sum_t \gamma^t r_t]$.
- In many tasks, **uncertainty and tail behavior** matter.
- Distributional RL models the full random return, not just its mean.
- This often improves stability and empirical performance.

Takeaway: Value distributions contain useful learning signal.

Quick RL Recap (for context)

We assume an MDP with state s , action a , reward r , discount $\gamma \in [0, 1)$.

- Policy: $\pi(a \mid s)$
- Classical Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

- DQN approximates $Q^*(s, a)$ with a neural network.

Return Distribution Instead of Mean

Define random return under policy π :

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Distributional RL models:

$$Z^{\pi}(s, a) \stackrel{D}{=} G_t \mid (S_t = s, A_t = a)$$

where $\stackrel{D}{=}$ denotes equality in distribution.

Expected value is recovered by:

$$Q^{\pi}(s, a) = \mathbb{E}[Z^{\pi}(s, a)]$$

Distributional Bellman Operator

The distributional Bellman update is:

$$(\mathcal{T}^\pi Z)(s, a) \stackrel{D}{=} R(s, a) + \gamma Z(S', A')$$

with $A' \sim \pi(\cdot \mid S')$.

Optimal-control version:

$$(\mathcal{T}Z)(s, a) \stackrel{D}{=} R(s, a) + \gamma Z(S', a^*(S'))$$

where $a^*(S') = \arg \max_{a'} \mathbb{E}[Z(S', a')]$.

Important: Contraction properties depend on the metric (e.g. Wasserstein/Cramer).

C51 (Categorical DQN)

Idea: represent each $Z(s, a)$ on fixed support atoms $\{z_i\}_{i=1}^N$.

- Network outputs probabilities over atoms per action.
- Bellman target distribution is projected back to fixed support.
- Training minimizes cross-entropy between predicted and projected target.

Pros: Simple, first major success.

Limitations: Fixed support can be restrictive.

QR-DQN and IQN

QR-DQN (Quantile Regression DQN)

- Learns quantile locations directly.
- Uses quantile Huber loss.

IQN (Implicit Quantile Networks)

- Learns a continuous quantile function $F_Z^{-1}(\tau)$.
- Samples $\tau \sim \mathcal{U}(0, 1)$ during training/inference.

Rule of thumb: QR-DQN and IQN are often stronger baselines than C51.

Why It Works in Practice (High-level)

- Richer target than scalar TD target.
- Better gradient signal from multiple quantiles/atoms.
- Improved robustness in stochastic environments.
- Natural bridge to risk-sensitive control.

Risk-sensitive Decision Making

Once we estimate $Z(s, a)$, we can optimize objectives beyond expectation:

- Mean (risk-neutral): $\mathbb{E}[Z]$
- CVaR / lower quantiles (risk-averse)
- Upper quantiles (risk-seeking)

Example objective:

$$a^*(s) = \arg \max_a \text{CVaR}_\alpha(Z(s, a))$$

Experiment Setup (TODO)

Fill this slide with your own setup.

- Environment(s): *e.g. CartPole, Atari, custom task*
- Algorithms compared: *DQN, C51, QR-DQN, IQN*
- Training budget: *steps, seeds, hardware*
- Metrics: *return mean, variance, worst-case score*

Results (TODO)

Replace with plots/tables.

- Learning curves
- Distribution snapshots over training
- Ablations (atoms/quantiles, learning rate, etc.)

References

- Bellemare, Dabney, Munos (2017): A Distributional Perspective on RL.
- Dabney et al. (2018): Distributional RL with Quantile Regression.
- Dabney et al. (2018): Implicit Quantile Networks for Distributional RL.